This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

PFQ: A Proactive Fair Queueing Scheme Ensuring Fairness and High Utilization in Data Center Networks

Yike Wang, Qing Li, *Senior Member, IEEE*, Feixue Han, Changlin Jiang, Yuan Yang, *Member, IEEE*, Rui Huang, Yong Jiang, *Member, IEEE*, Mingwei Xu, *Senior Member, IEEE*

***Abstract*—Modern data centers typically support multiple applications with diverse network demands. As a fair bandwidth allocation approach, Fair Queueing (FQ) manages to achieve performance isolation from the network side. However, the proposed approaches face intrinsic limitations in practice. Current schemes fail to achieve fairness and low drop rates simultaneously. Furthermore, they struggle to handle incast traffic. In this paper, we propose Proactive Fair Queueing (PFQ), which aims to achieve consistently high performance with limited priority queues, especially for incast traffic. More specifically, a buffer space reservation mechanism is used to better cope with challenging traffic patterns. PFQ dynamically recycles and reallocates buffers to improve link utilization while prioritizing fairness. Additionally, PFQ periodically adjusts the mapping of flows to physical queues, ensuring that packets are distributed more evenly across the queues. We conduct extensive experiments to evaluate PFQ's performance in NS3. The experiment results demonstrate that PFQ achieves both fair bandwidth allocation and high network utilization under different network conditions. Especially when incast occurs, PFQ reduces packet drop rate by up to 74.3% over the state-of-the-art fair queueing mechanisms.**

***Index Terms*—Fair queueing, Max-min fairness, Buffer utilization.**

I. Introduction

MODERN data centers have stringent requirements on switches. Besides line-rate forwarding, it is also very important to ensure fairness among flows, because without such fairness, the limited buffer can be easily exhausted by a few flows, *e.g.*, elephant flows or ill-behaving ones, leaving other flows with high loss rates and large latency. This hurts the network performance, especially for applications that require a small tail latency. Fair queueing (FQ) is a technique promising to provide per-flow fairness. Unlike end host-based methods such as congestion control (CC), FQ manages the output buffer at each port of a switch and determines the order to output packets [1]. There have been studies on FQ over decades [2]–[9], but few approaches have been used in commodity switches due to the cost of implementation. In recent years, with the advance of programmable switch technology [10]–[14], researchers began to revisit FQ and develop improved algorithms to cope with the traffic of emerging applications [15]–[20].

FQ aims to provide an illusion that every flow has its own queue, as such the flows are isolated, and the bandwidth of the port is shared fairly by all flows traversing it [3], [4]. Unfortunately, it is well known that the resources are expensive and limited in switches. Typically, commodity programmable switches support a maximum of just eight priority queues at each output port [21]–[24], which is much less than the number of possible concurrent flows. One intuitive approach is that FQ allocates the same size of buffer space to each flow. However, a natural problem arises: buffer resources cannot be sufficiently utilized since the size of traffic flows follows the heavy-tailed distribution [18], [19], [25]. For example, an elephant flow may easily exhaust the allocated buffer and suffer continuous packet drop, while many small flows are using the over-provided buffer that is under-utilized. Furthermore, fairness should also be provided to flows yet to come, especially for incast traffic and micro-bursts [26]. Overuse of buffer can bring severe packet loss to subsequent flows. Intrinsically, these problems are incurred by complex and rapidly changing network traffic, which makes the development of an ideal FQ a very challenging task.

Existing studies [15]–[20] do not fully address these issues. As one of the state-of-the-art approaches, Approximating Fair Queueing (AFQ) [17] allocates the packets from the per-flow "virtual" queue to limited physical resources and uses a round-robin method to closely simulate the output order of packets, but the virtual queue of each flow shares the same size of buffer, which fails to match the distribution of flow size and dynamically adapt to network changes. Another approach, Elastic Fair Queueing (EFQ) [18], [19] improves buffer utilization by allocating available buffer space to each active flow. However, little buffer space may be left for subsequent flows, and this compromises fairness, especially for bursty and incast traffic. There are also other approaches

Yike Wang, Feixue Han and Rui Huang are with the Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China. (e-mail: wyk23@mails.tsinghua.edu.cn; hanfx19@mails.tsinghua.edu.cn; huang-r24@mails.tsinghua.edu.cn).

Yong Jiang is with the Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China, and also with the Department of Strategic and Advanced Interdisciplinary Research, Pengcheng Laboratory, Shenzhen 518055, China. (e-mail: jiangy@sz.tsinghua.edu.cn).

Qing Li and Changlin Jiang are with the Department of Strategic and Advanced Interdisciplinary Research, Pengcheng Laboratory, Shenzhen 518055, China. (e-mail: liq@pcl.ac.cn; jiangchl@pcl.ac.cn).

Yuan Yang and Mingwei Xu are with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China. (e-mail: yangyuan_thu@mail.tsinghua.edu.cn; xmw@cernet.edu.cn)

Corresponding author: Qing Li

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

with various considerations [15], [16], [20] but they have not solved the challenges completely. As a result, there are still unnecessary packet loss, link under-utilization, and unfairness to a certain extent, which not only damage user experience but also degrades switch performance like throughput.

In this paper, we propose **P**roactive **F**air **Q**ueueing (PFQ), a novel approach that uses limited priority queues to achieve fair bandwidth allocations and high network utilization, for both the existing and uncoming flows, especially when incast occurs. The core of PFQ is to manage the buffer usage in a proactive manner, enabling better dynamic adaption to traffic changes. In particular, we develop three key measures. First, PFQ monitors the number of active flows and the flow number change, based on which PFQ reserves a portion of the buffer space to deal with possible incast traffic that may come in the future. Second, PFQ monitors the utilization of buffer allocated to active flows, based on which PFQ recycles buffer from under-utilized flows (*i.e.*, small or inactive flows) and reallocates the buffer to elephant flows. Thus, the overall resource utilization can be improved effectively. Third, PFQ maintains a mapping between flows and a limited number of virtual queues instead of allocating a virtual queue to each flow. Then, PFQ counts forwarded packets number of each flow in a period, and maps flows to virtual queues according to the results, so as to ensure a more balanced utilization of queues.

We implement PFQ and other state-of-the-art approaches in NS3. The simulation results show that PFQ can improve the fairness index by up to 17.5% and reduce packet drop rate by up to 74.3% in incast traffic. The main contributions are summarized as follows.

- We make in-depth analyses on the key challenges faced by FQ under real-world constraints such as limited priority queues. Based on the analyses, we show the limitations of existing FQ approaches, *e.g.*, AFQ and EFQ.
- We propose PFQ to overcome the challenges. We develop the mechanisms of buffer reservation, buffer recycling, and virtual queue mapping for PFQ to enable fair bandwidth sharing and high network utilization.
- We conduct extensive experiments using NS3 to validate our analyses and show the benefit of PFQ. Our comprehensive experiments show that PFQ achieves consistently high performance under a series of network conditions, especially when dealing with incast traffic.

II. Background and Motivation

In this part, we first present three key challenges in scenarios with limited FIFO queues. Based on them, we analyze the problems of the state-of-the-art FQ schemes, *e.g.*, AFQ and EFQ. Finally, we summarize and outline the direction of our work.

A. Key challenges

Current commodity switches [10]–[14] typically support up to eight priority queues [21]–[24]. In this scenario, the proposed FQs fail to achieve fairness and high buffer utilization simultaneously and maintain high performance constantly. They face the following three critical challenges.

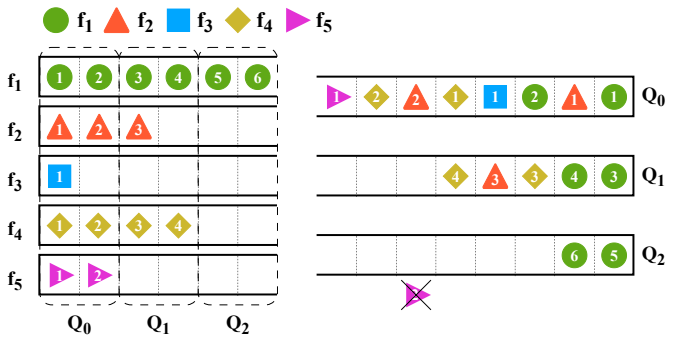


Fig. 1. An example of AFQ. On the left are ideal per-flow queues while three physical queues are on the right. The 2nd packet of flow f_5 is designated to enter queue Q_0. However, when queue Q_0 is already full, the packet will be discarded.

(i) Max-min Fairness. Max-min Fairness. Max-min fairness [27] allocates resources so that no flow's rate can be increased without decreasing the rate of some other flow that already has an equal or smaller share. In other words, it protects the most bandwidth-constrained flows and prevents any single flow from starving others. Fair queueing (FQ) is a natural mechanism for implementing max-min fair packet scheduling in data center networks [28], because it approximates per-flow service while sharing a limited number of physical queues. However, achieving max-min fairness in practice is challenging. Modern data centers host a wide variety of applications that generate heterogeneous flows, ranging from latency-sensitive short flows to throughput-oriented long flows. These flows start and stop frequently, and their sending rates evolve over time as congestion control reacts to network conditions. Under such highly dynamic and mixed workloads, an FQ scheme with limited queue and buffer resources must continuously adapt to diverse per-flow demands, making it difficult to keep the resulting bandwidth allocation close to the max-min fair ideal.

(ii) High buffer utilization & Low drop rate. The link speed in data center networks is rapidly increasing, while the buffer size of commodity switches lags behind. As a shared resource, data center network switches mostly have shallow buffers, up to tens of kilobytes per port per Gbps [29], [30]. To ensure that each flow can be allocated sufficiently, it is essential to improve the buffer utilization. However, the size of traffic flows follows the heavy-tailed distribution: around 10% of elephant flows occupy around 90% of network traffic, while about 90% of mice flows occupy around 10% of traffic [18], [19], [25]. Therefore, evenly allocating buffers ignores the characteristics of flows and only provides fair sharing among flows, which may result in high packet drop rates for elephant flows. Moreover, it is challenging to maintain high buffer utilization and low drop rate while achieving Max-Min Fairness. Ensuring fairness is the highest priority so we should not lose sight of the fairness in pursuit of utilization ratio and drop rate.

(iii) Robustness to incast. Incast occurs in data center

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

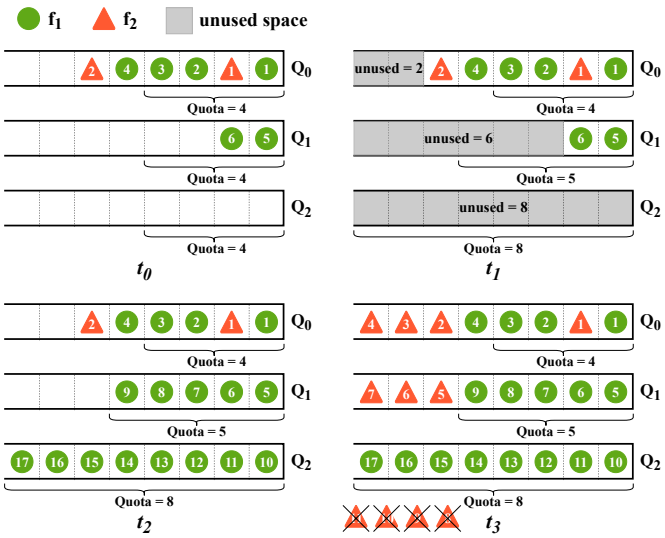


Fig. 2. An example of EFQ. When the five green packets of flow f_1 and the two orange packets of flow f_2 arrive at t_0, they enqueue into Q_0 and Q_1. At t_1, the queue's quota is reallocated. Hence, the flow f_1 that arrives first occupies more space than the flow f_2 that arrives later, leading to the unfairness of EFQ.

networks when multiple senders simultaneously send data to the same receiver [31]. The bottleneck switch buffer is over-filled quickly as the number of competing senders increases when incast occurs, leading to packet loss and, therefore, TCP throughput collapse, which substantially degrades the application's performance [32]. However, current FQs lack effective preventive measures to mitigate the impacts of incast. To effectively manage numerous partition and aggregation operations, FQ necessitates a flexible buffer reservation mechanism capable of supporting the incast of thousands of flows.

B. Diving into AFQ and EFQ

Although AFQ and EFQ have proven the feasibility of implementing FQ on programmable switching ASICs [17]–[19], they present several performance issues in practical deployments.

1) Unguaranteed fairness: Every flow in AFQ transmits a fixed number of bytes in each round, *i.e.* BpR bytes (bytes per round). On arrival, each packet is assigned a departure round number based on the number of bytes the flow has sent. Packets are scheduled to be transmitted in increasing round numbers by serving the physical queues in a round-robin manner. AFQ ensures fairness by assigning the same quota to each flow, but it does not guarantee that the allocated packets will be successfully enqueued.

We use Figure 1 to illustrate this problem. On the left are the ideal per-flow queues for calculating the departure round number of new packets for that flow. On the right are three physical queues, each with space for eight packets. For simplicity, we assume that all flows have the same packet sizes and each packet is MTU size. And we set the BpR to 2 MTUs. The packets from flow f_1 to f_4 are enqueued into the physical queue in order of calculated round numbers. Upon the arrival of the two packets from flow f_5, they are assigned to queue Q_0. However, since queue Q_0 has only one available

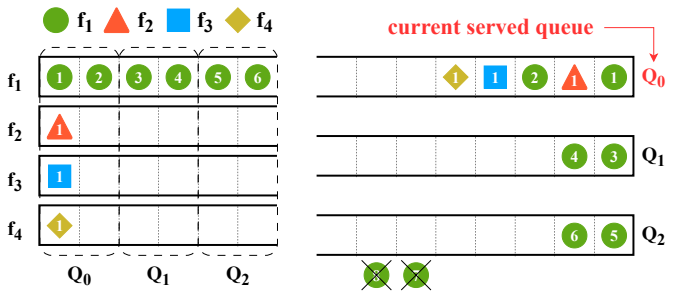


Fig. 3. Another example of AFQ. Mice flows with a higher proportion waste allocated buffer space, resulting in low buffer utilization. In addition, elephant flows experience unnecessary packet loss even when sufficient buffer space remains available in the queue.

MTU, the 2nd packet of flow f_5 will be discarded. The flow virtual queue only determines the physical queue to which a packet is assigned, but it cannot ensure that there is sufficient space for the packets scheduled to be enqueued. Such uneven distribution often occurs at the beginning of network operation or when the flows have roughly equal lengths.

EFQ introduces the concept of quota (Θ), and Θ determines the amount of data that EFQ allows each flow to put into the ith FIFO queue. To improve buffer utilization, EFQ adjusts Θ of each physical queue according to the residual buffer space to reduce packet loss. However, EFQ over-allocates unused buffers to large flows, which compromises the fairness of the scheme and results in unfair bandwidth allocation.

We use an example to explain this problem. Figure 2 shows an example of EFQ with three FIFO queues (i.e., Q_0, Q_1, and Q_2), and each queue also has eight packets of buffer space. Two active flows are competing for the buffer. At time t_0, EFQ evenly allocates four packets of space (i.e., $\Theta_0 = \Theta_1 = \Theta_2$ = four packets) to each flow per queue. Six packets of flow f_1 and two packets of flow f_2 arrive successively. In this case, Q_0, Q_1, and Q_2 have two, six, and eight packets of unused buffer spaces, respectively. The quotas are adjusted by reallocating the unused buffer space at t_1. Concretely, Θ_1 and Θ_2 increase to five and eight packets, respectively. At t_2, when eleven packets of flow f_1 arrive, they enter the Q_1 and Q_2 according to the quotas. After that, the buffer has only five available MTUs. Hence, four packets will be discarded when the nine packets of flow f_2 arrive at t_3.

Specifically, we can find that the total quotas of two flows reach thirty-four MTUs, which is more than the twenty-four MTUs of buffer space. An excessive quota causes the flow that arrives first to unfairly occupy the space of the flow that arrives later, thus taking up more buffer space (for example, flow f_1 occupies seventeen MTUs while flow f_2 only occupies seven MTUs). Therefore, the over-allocation of EFQ can not guarantee fairness.

2) Unnecessary packet drop: AFQ calculates a departure round number for each packet on arrival. It is worth noting that if the departure round number exceeds the threshold (*departure round - current round* $\geq$ *number of physical queues*), the packet is discarded; otherwise, it enqueues into the queue corresponding to the departure round number. When encountering mixed flows with a long-tailed distribution, AFQ

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

will experience low buffer utilization and the high drop rate.

Figure 3 illustrates this problem with an example. Flow f_1 simulates an elephant flow while flows f_2, f_3, and f_4 simulate mice flows. We still assume per-packet size is MTU and the current served queue is Q_0. AFQ allocates the same amount of buffer to each flow in the physical queue to achieve fairness. On the one side, mice flows do not have enough packets to fill the space and only occupy a small amount of the allocated buffer. The space allocated to mice flows f_2, f_3, and f_4 in both queues Q_1 and Q_2 is wasted. To make matters worse, the mice flows dominate the data center traffic [18], [19], [25], further reducing buffer utilization. On the other side, elephant flows experience unnecessary packet loss, even though there are available buffer in the queue. Newly arriving packets (i.e., 7th and 8th packets) of f_1 are still dropped despite the availability of Q_1 and Q_2, leading to a high drop rate.

While EFQ enhances buffer utilization by adjusting the Θ values of different physical queues, it does not effectively reduce packet loss rates across all elephant flows. In particular, large flows that arrive earlier tend to preempt buffer space, resulting in substantial packet loss for flows that arrive later. As shown in Figure 2, the flow f_2 that arrives later suffers packet loss due to its allocated space being occupied by f_1 that arrives earlier. Generally, improving buffer utilization at the cost of fairness is not a worthwhile trade-off.

3) Poor performance for incast: AFQ and EFQ both experience significant performance degradation when the incast occurs. This issue arises from two key causes. On the one hand, neither FQ has a buffer space reservation mechanism. Neither of the FQs accounts for potential future traffic. When plenty of new flows arrive, the buffer has been filled with packets from old flows, leading to the packet drop of the new arrivals. On the other hand, AFQ and EFQ are both inherently limited in their ability to allow a large number of flows to coexist due to their rigid allocation methods. If we have nQ queues in the system and define the average round-trip latency of the data center network as d and the peak line rate as l, then the BpR of AFQ is fixed to $\frac{d*l}{nQ*6}$ [17]. Consequently, even when the buffer is available, the number of coexisting flows remains constant to $\frac{B}{nQ*BpR}$ (B is the total buffer size). We use the Arista 7010T (total buffer size is 4MB) [29] as an example. AFQ can only support about 240 flows for a 40 Gbps link, 20 us RTT, and 8 FIFO queues. For EFQ, the allocated buffer space of each flow is divided equally among each physical queue, resulting in a small value of Θ. As new flows arrive, Θ gradually decreases. To ensure that Θ does not fall below the MTU size, EFQ only supports a limited number of flows, even when the buffer is empty.

C. Summary

We advocate that the ideal FQ for data center networks should maintain high buffer utilization and low drop rate while prioritizing Max-min Fairness, which is critical for achieving fair bandwidth distribution and ensuring resource efficiency. In addition, efficient countermeasures are required to address incast traffic scenarios. AFQ achieves fairness theoretically, but the imbalanced packet allocation limits its practical implementation. Moreover, it falls short in buffer utilization and incast performance. As for EFQ, it excels in buffer utilization but struggles with fairness and incast handling. To meet all three challenges, we propose and implement PFQ, which achieves the desired FQ properties simultaneously.

III. Design

In this section, we first describe the design insight that inspires the operating principle of PFQ in Section III-A. Next, we outline the overview of PFQ in Section III-B. More precisely, to understand how PFQ can address the three key challenges discussed above, we analyze the allocation buffer space in Section III-C and discuss its queue management in Section III-D.

A. Design Insight

In programmable switches, the number of queues per port is limited (no more than eight), primarily due to cost constraints. Due to the limited hardware resources of programmable switches, existing schemes exhibit several shortcomings in terms of buffer and queue management.

(i) The approach of allocating the entire buffer does not account for future network demands. Two potential issues may arise: (1) the inability to manage complex traffic patterns (*i.e.*, incast traffic) leads to performance degradation, and (2) when existing flows completely occupy the buffer, newly arriving flows may experience packet loss. Therefore, instead of allocating the entire buffer, FQ should reserve a portion of it to accommodate future traffic demands.

(ii) Strict fairness leads to inefficient utilization of buffer resources. Given the long-tailed distribution of flow sizes, the even allocation of buffer leads to considerable waste for mice flows. Current FQs initially allocate the same amount of buffer to each flow to prioritize fairness. Due to the varying sizes of flows, small flows often leave a large amount of buffer underutilized. Moreover, the network environment is continuously changing due to the varying flow sizes. To dynamically adapt to network conditions, one method for FQ is reallocating a portion of buffer space from inactive or small flows to larger ones, which can significantly improve buffer utilization and reduce packet loss rate.

(iii) Per-flow virtual queues leads to uneven distribution. The previously proposed FQ allocates a dedicated virtual queue for each individual flow. However, due to the heavy-tailed nature of flow size distributions in the network, packets entering the physical queue tend to cluster in a few specific queues, leading to an uneven packet distribution across the physical queues. This unevenness becomes more problematic when one queue receives an excessive number of packets, potentially causing unnecessary packet loss. Further details will be discussed in Section III-D1. To address this issue and achieve a more balanced distribution, we suggest maintaining a fixed number of virtual queues while periodically remapping flows to these queues.

Grounded in the shortcomings discussed above, we propose PFQ, which optimizes the buffer management by periodically executing *recycle-merge-reallocate* process to improve link

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

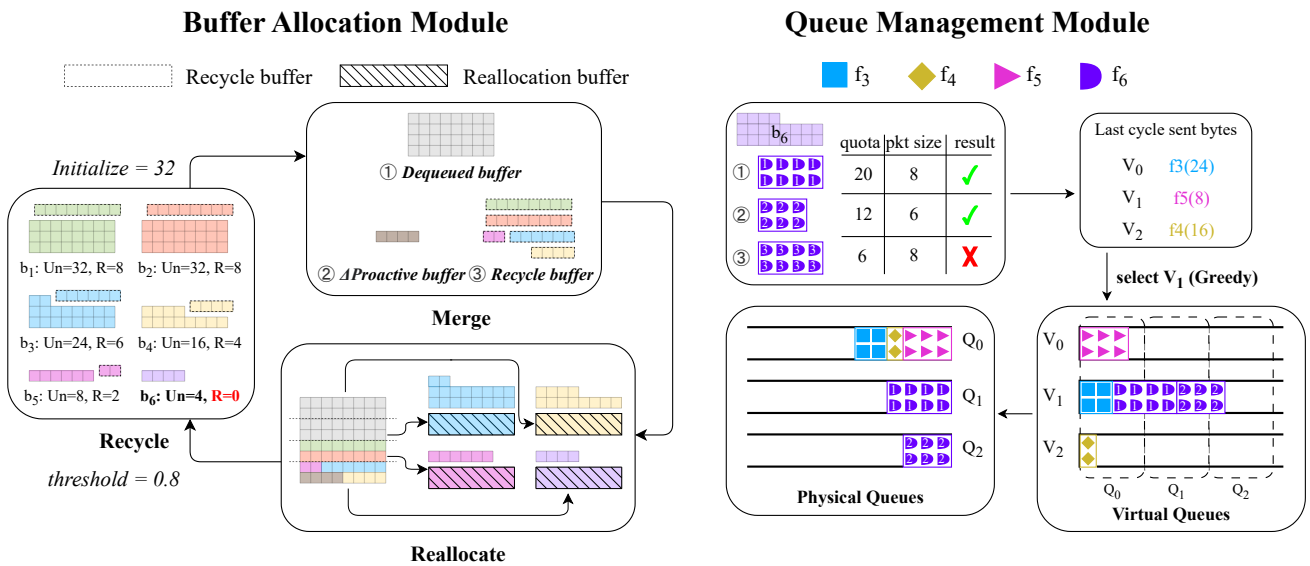


Fig. 4. The workflow of PFQ. On the left is the buffer allocation module, which consists of a cycle of *recycle-merge-reallocate*. Assume that each flow is initially allocated a buffer size of 32. Un and R represent the sizes of the buffers that are unused and recycled, respectively. Given a recycling threshold of 0.8, all flows, except flow f_6, are required to recycle their buffers. PFQ merges three components: *Dequeued buffer*, Δ*Proactive buffer* and *Recycle buffer*. On the right is the queue management module, where packet ③ is dropped after the comparison between the quota of b_6 and the packet size. Given that packet ① and ② fall within the intervals of Q_1 and Q_2 in the virtual queue, they are subsequently directed to the corresponding physical queues.

utilization while prioritizing fairness and introduces *queue-mapping* mechanism to improve the efficiency of queue management. The detailed design of the PFQ is as follows.

B. PFQ Overview

Figure 4 presents the core design of PFQ. On the left is the buffer allocation module. A certain portion of the buffer will be reserved and excluded from allocation, termed as *Proactive buffer*, the size of which is determined by flow number and time (the calculation will be explained in Section III-C). Intuitively, the buffer of inactive flows or mice flows is under-utilized. To this end, we propose the buffer usage ratio threshold. PFQ emphasizes simplicity and efficiency by using statically tuned thresholds, thereby avoiding the complexity and potential overhead associated with an adaptive control loop. If the usage ratio of the allocated buffer is lower than the threshold, PFQ recycles a portion of this flow's buffer (the area highlighted by the dashed line) to increase buffer utilization. Merged with the size of the *Dequeued buffer* (the gray area) and the variation in the *Proactive buffer* between the current cycle and the last cycle (the brown area), and the combined space is evenly distributed among all active flows. By repeating the above *recycle-merge-reallocate* process, the buffer flows from inactive flows to active flows.

On the right is the queue management module, where packets are scheduled to enqueue. We take flow f_6 in Figure 4 as an illustrative example. Upon the arrival of a data packet, it is essential to first compare the remaining buffer allocation for the current flow with the size of the incoming packet to determine whether to accept the packet. After accepting packet ① and ②, f_6 lacks sufficient space to accommodate packet ③, resulting in the discard of packet ③. To ensure an even distribution of packets in the physical queues, PFQ maps the flows to a fixed number of virtual queues, where the virtual queue is determined based on the packets transmitted by the flow in the previous cycle using the Greedy algorithm. For example, PFQ maps f_6 to the virtual queue V_1 because the length of V_1 is currently the shortest. Finally, the choice of the physical queue is determined by the current length of the virtual queue.

Our key innovation directly addresses the shortcomings of the existing schemes mentioned in Section III-A, where lies in ***(i) Proact***: *reserving a portion of the buffer ensures better accommodation of future demands in the buffer allocation module.* ***(ii) Recycle***: *reclaim buffers from small or inactive flows and reallocate them to large flows to improve overall resource utilization.* ***(iii) Map***: *periodically mapping flows to virtual queues helps to achieve a more balanced distribution of packets into the physical queues within queue management.*

C. Buffer Allocation Module

The pseudocode for the buffer allocation module is shown in Algorithm 1. When considering buffer allocation, we count all priority queues as a whole. The buffer allocation updates at fixed intervals of the time duration T (line 2). The total available buffer in this cycle is strictly less than the total physical buffer size and consists of the following three parts:

(1) ***Dequeued buffer*** (*totDeq*): the total size of packets dequeued in the previous cycle (line 3). The space previously occupied by the dequeued packets is now freed up, where this part of the buffer is allowed for enqueuing new packets in the next cycle.

(2) Δ ***Proactive buffer*** (Δ *totPro*): the changing size of the proactive buffer (line 4). The proactive buffer space, which is guaranteed to be ready for unforeseen circumstances, adjusts dynamically in response to network conditions. The size of the proactive buffer is determined by the two dimensions of flow

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

Algorithm 1: Buffer Allocation Module
parameters: $state_i$: the state of ith flow
N : # of active flows
totBuffer : total available buffer in this cycle
totDeq : total dequeue packet size in the last cycle
$quota_i$: the bytes can be sent by i^{th} flow

```
function Buffer_Allocation_Update()
    while Timer expires after T do
        totBuffer += totDeq
        totBuffer += Δ totPro
        totBuffer += totRec
        for i in state.length() do
            if state_i == True then
                N++
        for i in state.length() do
            if state_i == True then
                quota_i += totBuffer / N
        totBuffer = totDeq = N = 0
```

number and time. By default, PFQ leverages the following function to calculate the proactive buffer space:

$$totPro = min(B \cdot f(N) \cdot g(N, \hat{N}), B * 0.5) \tag{1}$$

where B is the total switch buffer space, N refers to the number of active flows in the previous cycle, $\hat{N}$ refers to the number of active flows in the cycle before the previous one, $f(N)$ is a flow number influencing function and $g(N)$ is a time influencing function, respectively. To ensure the proactive space does not become excessively high, a maximum value of $B * 0.5$ is enforced. Specially, $f(N)$ is defined as follows:

$$f(N) = \frac{\alpha}{\alpha + N} \tag{2}$$

where the $\alpha > 0$. In Section IV-A, we list the values of all hyperparameters. The function $f(N)$ is determined exclusively by the number of active flows. Generally, when the value of N in the network is relatively small, $f(N)$ tends to be larger, which indicates that a larger proactive buffer space is required, allowing for additional room to be allocated in the future. As more flows arrive (N increases), the proactive space in PFQ gradually decreases. It's not hard to understand: when there are 99 flows in the network, only $\frac{B}{100}$ of the space needs to be reserved to ensure fairness for the next incoming flow while the value jumps to $\frac{B}{3}$ in a scenario with just two flows.

Another function, $g(N, \hat{N})$, affects $totPro$ in the time dimension. It is defined as follows:

$$g(N, \hat{N}) = e^{\frac{\beta}{|N - \hat{N}| + 0.5}} \tag{3}$$

where the $0 < \beta < 1$. PFQ assesses the network condition based on the variation in the number of active flows, thereby determining whether to store or release the proactive buffer. In particular, when the network reaches a stable state, *i.e.*, N remains similar over two consecutive cycles, PFQ increases $g(N, \hat{N})$ properly to cope with the subsequent situation. Conversely, in a complex and unstable network environment, it

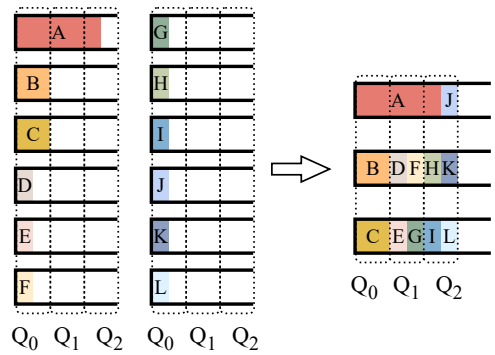


Fig. 5. The comparison between the virtual queues. On the left is the design of the previously proposed FQ, while on the right is the architecture of the PFQ.

is appropriate to reduce the allocation of $totPro$ accordingly, which indicates that PFQ has detected the challenging traffic pattern (*i.e.*, incast traffic), and therefore has released the proactive buffer to address it.

Finally, we denote that $totPro$ refers to the total size of the proactive buffer in the current cycle and $\hat{totPro}$ represents the proactive buffer size from the previous cycle. We have:

$$\Delta totPro = totPro - \hat{totPro} \tag{4}$$

Clearly, $\Delta totPro$ could potentially be a negative value.

(3) ***Recycle buffer*** (*totRec*): the total size of packets recycled from flows (line 6). To improve buffer utilization, PFQ recycles a portion of the buffer proportionally based on the allocated buffer and the actual usage for each flow. Firstly, we calculate the buffer utilization ratio of ith flow allocated in the previous cycle:

$$ratio_i = \frac{Sent_i}{Allocated_i} \tag{5}$$

where $Sent_i$ and $Allocated_i$ represent the packet size transmitted by i^{th} flow and its allocated buffer size during the previous cycle, respectively. Secondly, we use the following function to calculate the recycle buffer for each flow:

$$Rec_i = \begin{cases} 0, & \text{if } ratio_i \geq threshold \\ \frac{1}{\gamma} \cdot e^{-\gamma \cdot ratio_i}, & \text{if } 0 \leq ratio_i < threshold \end{cases} \tag{6}$$

where the $\gamma > 1$ and $threshold$ is the boundary of the recycle function. When the buffer utilization ratio for a flow in the previous cycle exceeds $threshold$, we consider that it has a significant demand for buffer resources. Consequently, PFQ does not recycle buffer resources from these flows. For other flows, there is no demand to occupy the remaining buffer space. We recycle a portion of the buffer and allocate it to flows with greater needs, thereby minimizing waste. Specifically, we use the exponential power of e to ensure that Rec_i increases as $ratio_i$ decreases correspondingly.

Thirdly, the total size of the recycle buffer is calculated as follows:

$$totRec = \sum_i Rec_i \cdot quota_i \tag{7}$$

where $quota_i$ refers the bytes can be sent by i^{th} flow.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

D. Queue Management Module

In this section, we focus on workflow concerning the arrival and transmission of data packets in PFQ.

Algorithm 2: Mapping
parameters: V : # of virtual queues
$enq_idx[i]$: index of ith flow enqueuing virtual queue
$totLast[i]$: total size of packets send by the flow mapped to the ith virtual queue in the last cycle
$Last_sent[i]$: total size send by i^{th} flow in the last cycle

```
1  function Mapping(pkt)
2      while Timer expires after T do
3          if enq_idx[pkt.fid] ≠ -1 then
4              return
5          else
6              cur_idx = -1
7              cur_min = DOUBLE_MAX
8              // Find the shortest virtual queue
9              for i in V do
10                 if totLast[i] < cur_min then
11                     cur_idx = i
12                     cur_min = totLast[cur_idx]
13             // update value
14             enq_idx[pkt.fid] = cur_idx
15             totLast[cur_idx] += Last_sent[pkt.fid]
```

1) Mapping algorithm: Figure 5 illustrates the differences in the design of virtual queues between PFQ and other proposed FQ schemes. PFQ sets a fixed number of virtual queues independent of the number of flows. PFQ maps flow to virtual queues, grouping, and processing different flows collectively, which alleviates packet imbalance in physical queues. The primary function of virtual queues in PFQ is to ensure fair and efficient distribution of packets across physical queues based on flow characteristics from the previous cycle. While more complex metrics could improve performance, PFQ prioritizes simplicity and efficiency by using a straightforward mapping approach, which balances fairness and resource usage while minimizing overhead. The flow-to-virtual-queue mapping scales efficiently, as it uses a fixed number of virtual queues (much smaller than the number of flows). We present the pseudocode of the mapping algorithm in Algorithm 2. Upon the arrival of a packet, it first checks whether the flow has already been mapped during the current cycle (line 2). If the flow has not been mapped to the corresponding virtual queue, PFQ will apply the **Greedy algorithm** (lines 8-12). Based on the sending packet size from the previous cycle, the flow is mapped to the shortest available virtual queue. All packets of this flow will enter the newly mapped virtual queue in this cycle. Note that the variable $enq_idx[pkt.fid]$ is initialized to -1 by default, indicating that no flow has been assigned to a virtual queue before the mapping algorithm is executed. Once a flow is mapped to a virtual queue, the corresponding index in $enq_idx[pkt.fid]$ is updated to reflect the assigned queue's index. This initialization step prevents ambiguity and ensures that flows are accurately assigned to virtual queues during the mapping process.

Algorithm 3: Enqueue & Dequeue
parameters: $state_i$: the state of ith flow
M : # of priority FIFO queues
R : current round number
BpR : bytes sent in each round
$enq_idx[i]$: index of virtual queue for i^{th} flow
$cur_len[i]$: current length of ith virtual queue
$quota_i$: the bytes can be sent by i^{th} flow
$Sent_i$: total size send by i^{th} flow in the current cycle
totDeq : total dequeue packet size in the last cycle

```
function Enqueue(pkt)
    state_pkt.fid = True
    if Sent_i + pkt.size() > quota_pkt.fid then
        drop(pkt)
        return false
    else
        Mapping(pkt)
        // find the length of the mapping virtual queue
        cur_len[enq_idx[pkt.fid]] += pkt.size()
        pkt_round = cur_len[enq_idx[pkt.fid]] / BpR
        // enqueue and update
        enqueue(pkt_round % M, pkt)
        Sent_pkt.fid += pkt.size()

function Dequeue()
    while True do
        if ! buffer.empty() then
            deq_idx = R % M
            // Drain deq_idx's queue till empty.
            while queue[deq_idx].length() > 0 do
                pkt = dequeue_pkt(deq_idx)
                send(pkt)
                totDequeue += pkt.size()
            // update the current round
            R = R + 1
```

Due to the continuous fluctuations in network conditions, the characteristics of flows are constantly changing. Based on the packet size of each flow in the previous cycle, PFQ updates the mapping at regular intervals of time T to better adapt to dynamic network changes. By utilizing the virtual queue length from the previous round, the mapping process is simplified, thus avoiding the computational overhead associated with real-time state tracking. This method also contributes to more stable and predictable scheduling, as reliance on real-time state fluctuations could lead to instability and inefficiencies. Furthermore, using the previous round's state helps mitigate short-term traffic variations, ensuring more efficient resource

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

allocation without the need for continuous re-evaluation.

2) Enqueue & Dequeue: The pseudocode for the enqueue and dequeue modules is shown in Algorithm 3.

Enqueue Module. The enqueue module determines the FIFO queue to which each packet will be assigned. On receiving a packet, the enqueue module mainly performs the following steps:

(1) PFQ compares the remaining allocated buffer of the flow with the size of the incoming packet (lines 3-5). If there is insufficient space, the packet will be dropped, and PFQ will wait for the next packet. Otherwise, move on to the next steps.

(2) PFQ determines the appropriate virtual queue and computes the corresponding physical queue for entry (lines 6-9). After finding the virtual queue with the function $Mapping()$, we add the size of the packet to its length (*i.e.* $cur_len[enq_idx[pkt.fid]]$). The packet's departure round number is computed as the current length divided by BpR.

(3) The packet enters the queue after the modulo calculation (line 11). Moreover, PFQ updates the parameters to ensure the correct execution of other modules (lines 12-14).

Dequeue Module. The dequeue module determines which FIFO queue to be serviced. It performs the following three steps:

(1) PFQ checks whether the buffer is empty (lines 17-18). If a packet exists, proceed to the next steps.

(2) PFQ determines the dequeue queue and drains it till empty (lines 19-24). Since the enqueue module has already assigned each packet a calculated departure round, PFQ must prioritize draining the queue with the smallest round number (i.e., the highest priority) in sequential order. Meanwhile, PFQ updates the parameter $totDeq$ specifically for the buffer allocation module.

(3) PFQ updates the current round (line 26). PFQ tries to approximate FQ by transmitting packets in a round-robin manner. PFQ arranges all physical queues in strict priority, assigning the highest priority to the queue with the lowest round number. Once empty, the queue should be demoted to the lowest priority and the current round number increments by one.

IV. Evaluation

In this section, we use NS3 simulator [33] to comprehensively evaluate the performance of PFQ. We first introduce the setup of our simulation, and then we compare the performance of PFQ with the state-of-art schemes under various scenarios. Next, we conduct ablation experiments to validate the effectiveness of the recycling module and virtual queues. Finally, we explore the impact of various parameters on the performance of PFQ.

A. Setup

Topology. In order to conduct comprehensive simulation evaluations, we select three classic topologies: fat-tree topology, leaf-spine topology [34], and dumbbell topology. In the experiments to evaluate overall performance (Section IV-B), we employed a fat-tree topology and a leaf-spine topology, while a dumbbell topology was used in the remaining experiments.

- **Leaf-Spine topology**: We construct an $4 * 4$ Leaf-Spine topology with 480 hosts, where each leaf switch is connected to 120 servers. The end-to-end propagation delay is 10µs, and the link bandwidth is 10 Gbps. Each switch has a fixed buffer size of 1MB per port, with 8 FIFO queues per port.
- **Fat-tree topology:** We implement the three-tier fat-tree topology, which composes 4 core switches, 8 aggregation switches, and 8 ToR switches. All switches are connected via 20Gbps links with a 10µs delay. Each ToR switch connects to 32 hosts, and a total of 256 hosts are included in this fat-tree topology. The links between each host and its corresponding ToR switch provide a bandwidth of 1Gbps and a delay of 100µs.
- **Dumbbell topology:** We use the dumbbell topology and the link between the two switches constitutes the bottleneck link. Each switch is connected to 32 hosts, which serve as the senders and the receivers, respectively. The settings of the link bandwidth and the end-to-end round trip latency will be detailed in the following experiments.

Comparison Schemes. We compare PFQ with three other FQ schemes. It is worth noting that Stochastic Fair Queueing (SFQ) hashes each packet into the corresponding queue on packets arrival. In the following experiments, each switch port is equipped with 8 FIFO queues and has an 8 MB buffer.

- **SFQ:** We implement SFQ following [5] with Deficit Round Robin (DRR) [6] on all switch ports. Furthermore, we periodically perturb the hash function.
- **AFQ:** We implement AFQ following [17] with a count-min sketch of size 2x16384. Also, the BpR is set to 1 MSS.
- **EFQ:** We implement EFQ following [18], [19]. The Flow-Aware Buffer Allocation (FBA) mechanism is added, and we set the fairness-efficiency factor α as 0.65.
- **PFQ:** We implement PFQ following the design described in the Section III. The values of α, β, and γ for PFQ are set to 200, 0.7, and 1.4, respectively. The threshold of the recycling function, $threshlod$, is set to 0.8. We set the time interval T to 20 ms.

B. Overall Performance

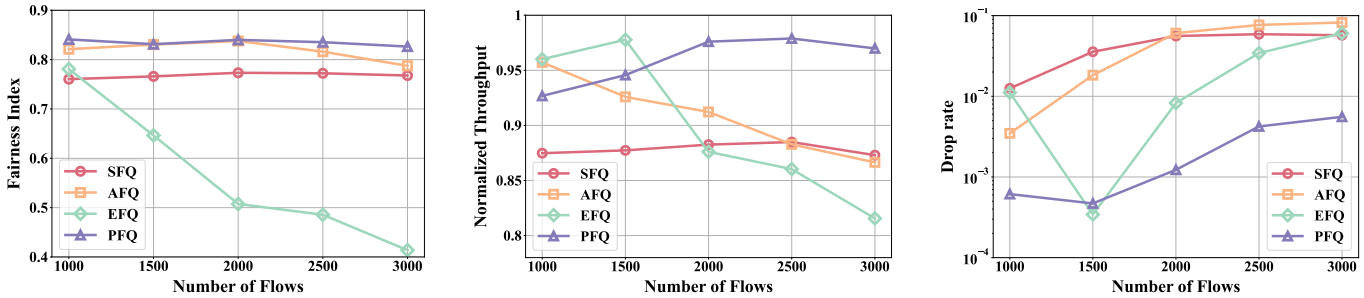
In this section, we evaluate short RPC flow (64 KB) completion time in a 31-to-1 incast scenario at 10 Gbps in a dumbbell topology, where the short flow is injected 10 ms after the incast onset. As shown in Table I, PFQ achieves a median FCT of 101.5 μs, significantly outperforming all baselines:

1) It reduces median FCT by 51.8% compared to FIFO + DCTCP (210.6 μs) and by 39.7% compared to FIFO + DCQCN (168.3 μs)—two representative ECN-aware and credit-based congestion control protocols widely used in production data centers.
2) Among fair queueing schemes, PFQ improves upon EFQ (134.2 µs) by 24.4%, AFQ (156.7 μs) by 35.2%, and SFQ (182.4 μs) by 44.4%.

This performance gap stems from fundamental architectural differences. DCTCP and DCQCN rely on reactive feedback:

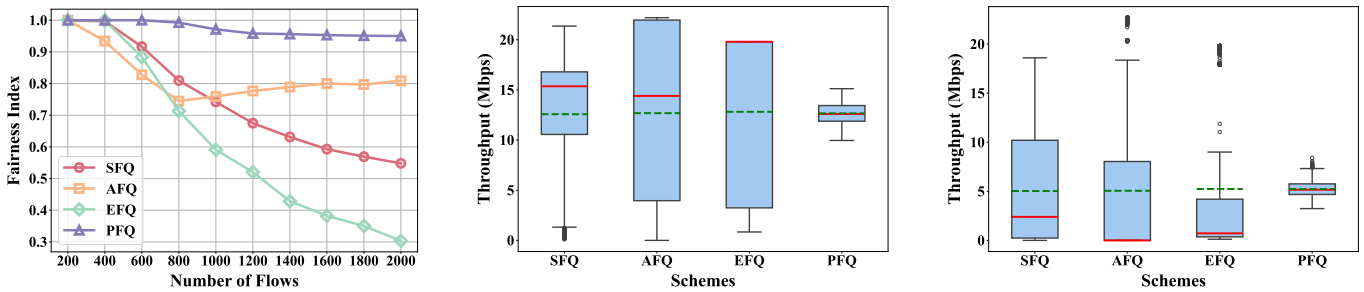
Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564



(a) Fairness Index (b) Normalized Throughput (c) Drop rate

Fig. 6. Overall Performance with large-scale network topology.



(a) Fairness Index (b) Box-Plot of 800 flows (c) Box-Plot of 2000 flows

Fig. 7. Performance of Fairness. In (b) and (c), the red solid line represents the median value of the throughput, while the green dashed line indicates the mean value of the throughput.

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT FAIR QUEUEING SCHEMES

Scheme	Median FCT (μs)	p99 FCT(μs)	Drop Rate
FIFO+DCTCP	210.6	485.2	35.1%
FIFO+DCQCN	168.3	392.7	28.4%
SFQ	182.4	315.6	12.3%
AFQ	156.7	278.9	8.7%
EFQ	134.2	260.3	21.5%
PFQ	**101.5**	**212.8**	**4.9%**

ECN marking or credit updates occur only after buffer occupancy exceeds thresholds, which is too slow to protect microsecond-scale RPCs during sudden incast bursts. Consequently, they suffer high short-flow drop rates (35.1% for DCTCP, 28.4% for DCQCN), leading to retransmissions and tail latency inflation. In contrast, PFQ provides proactive, in-network isolation: its buffer reservation mechanism guarantees space for newly arriving flows, while dynamic flow-to-queue remapping ensures balanced scheduling. As a result, PFQ maintains a low short-flow drop rate (4.9%) and consistently low queuing delay. Notably, even EFQ—despite its elastic buffer allocation—fails to protect short flows because it overcommits buffer to early elephant flows, leaving insufficient headroom for later arrivals. PFQ avoids this pitfall by decoupling fairness from instantaneous buffer occupancy and instead reserving capacity based on flow dynamics.

Next, we evaluate the performance of PFQ in the fat-tree topology. The flow size is evenly distributed among [2KB, 1MB]. The traffic load is 0.6. We conduct experiments in scenarios with five different numbers of flows, and the evaluation results are shown in Figure 6.

Figure 6(a) shows the *fairness index*, where we use the Jain Fairness Index [35] to quantify fairness with the following expression (the more approximate the value is to 1, the fairer it will be):

$$F = \frac{(\Sigma_i x_i)^2}{n \cdot \Sigma_i {x_i}^2} \tag{8}$$

where x_i denotes the throughput of i-th flow and n is the number of flows.

Figure 6(a) illustrates that PFQ consistently achieves a fairness index of approximately 0.83 across all five scenarios. In contrast, EFQ ensures fairness only in the scenario with 1000 flows. As the number of flows increases, however, the continuous enqueueing of packets from larger, earlier-arriving flows leads to packet loss for subsequent flows, resulting in a significant decline in the fairness index.

Figures 6(b) and 6(c) illustrate the *normalized throughput* and *drop rate*, respectively. Two figures show that both AFQ and SFQ exhibit relatively high packet loss, therefore, their throughput declines accordingly. As a fair queuing scheme that intentionally trades off fairness to achieve higher buffer utilization, EFQ demonstrates superior performance when the number of flows is relatively small (e.g., #flows = 1500). In such scenarios, the buffer is capable of accommodating packets from large flows, thereby yielding higher throughput and lower packet loss rates. However, with the increase in the number of flows, the buffer becomes incapable of accommodating additional packets, leading to significant packet loss and a noticeable decline in throughput. On the contrary, PFQ maintains consistently high throughput and effectively maintains a low packet loss rate even when the number of flows reaches 3000,

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

TABLE II
FAIRNESS, TAIL LATENCY, AND DROP RATE COMPARISON

Scheme	Jain's Fairness	p99 FCT (ms)	Drop Rate (%)
SFQ	0.32	124.6	7.9%
AFQ	0.51	85.3	3.8%
EFQ	0.44	98.5	5.2%
PFQ	**0.76**	**62.4**	**1.18%**

which is mainly contributed to the mechanism of proactively reserving buffer space.

To evaluate PFQ's resilience under the most challenging yet realistic conditions, we design a unified workload that combines adversarial on/off burst flows with heterogeneous RTTs and packet sizes. Specifically, 50% of flows behave maliciously—remaining minimally active to trigger buffer recycling and then launching large bursts—while concurrently exhibiting mixed RTTs (10 µs and 200 µs) and packet sizes (1.5 KB and 9 KB). As shown in Table II, PFQ maintains a high fairness index of 0.76, significantly outperforming EFQ (0.44), AFQ (0.51), and SFQ (0.32). This robustness arises from three design pillars: (i) byte-granular per-flow quota enforcement ensures that adversarial bursts—regardless of RTT or packet size—cannot exceed their dynamically adjusted fair share; (ii) the proactive buffer reservation absorbs surges from heterogeneous bursts without sacrificing isolation; and (iii) periodic virtual queue remapping prevents jumbo-frame bursts from concentrating in a single physical queue, thereby avoiding local tail-drop. In contrast, SFQ's hash-based FIFO sharing allows adversarial bursts to evict benign packets indiscriminately, while AFQ's fixed BpR misinterprets jumbo frames as multiple rounds, causing unfair scheduling. EFQ, though adaptive, reallocates recycled buffer to active adversarial flows without RTT-awareness, disproportionately harming long-RTT benign flows. Consequently, PFQ reduces the 99th-percentile flow completion time by 2.0× over SFQ, 1.5× over EFQ, and 1.3× over AFQ, while keeping drop rates below 1.2%. These results confirm that PFQ's switch-side isolation mechanisms provide strong fairness guarantees even when traffic is both diverse and adversarial—a critical property for multi-tenant data centers.

C. Fairness

In this experiment, we generate multiple flows simultaneously and evaluate whether they can share the bandwidth fairly. The bottleneck bandwidth is set to 10Gbps and the end-to-end round trip latency is 10µs. The arrival pattern still follows a Poisson distribution. We use two metrics to evaluate fairness: Jain Fairness Index [35], and the dispersion of flow throughput.

Figure 7(a) shows the *fairness index* and demonstrates that PFQ consistently maintains high fairness, which can be attributed to its prioritization of fairness during the buffer allocation process. As the number of flows increases, both SFQ and EFQ experience a sharp decline in fairness index, among which EFQ shows a more significant decrease. The fairness index of AFQ remains around 0.8, yet it still lags far behind PFQ.

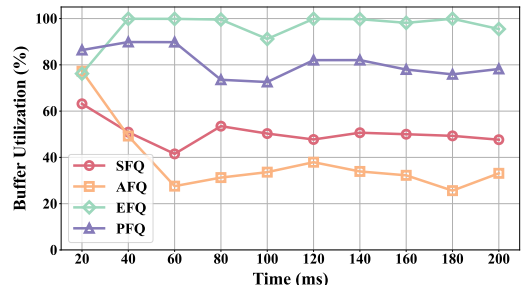


Fig. 8. Instantaneous buffer utilization in 10Gbps network.

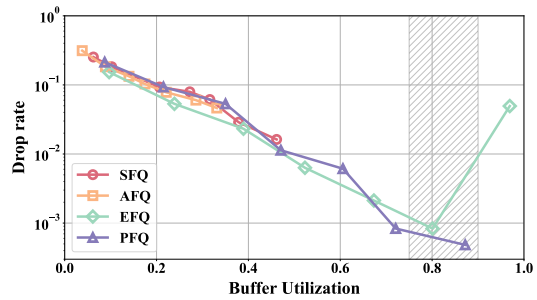


Fig. 9. The relationship between buffer utilization and drop rate. Each line contains seven points, which represents the simulation values under different traffic loads (0.1, 0.25, 0.4, 0.55, 0.7, 0.85, and 1).

To examine the dispersion of flow throughput, we use box plots to statistically analyze scenarios with 800 and 2000 flows, and the simulation results are shown in Figure 7(b) and 7(c), respectively. PFQ demonstrates commendable performance in ensuring fair bandwidth allocation. It is important to clarify that the fairness among flows is mainly managed by the Buffer Allocation Module. The Buffer Allocation Module determines whether a packet can be queued based on the flow's allocated buffer, ensuring fair and proportional resource distribution across active flows. Due to the randomness in packet enqueueing, the throughput distribution of SFQ flows is much more dispersed. When a large number of flows arrive simultaneously and AFQ is applied, the flows tend to concentrate in a few queues. The intense competition for the limited buffer resources results in extremely low throughput for many flows (the median value of throughput in Figure 7(c) is close to 0). As for EFQ, the majority of throughput is concentrated in a few flows, for the mean value remarkably surpasses the upper quartile with 2000 competing flows. This uneven allocation results in a significantly low Jain Fairness value for EFQ.

D. Buffer Utilization & Drop Rate

In this part, we first evaluate the buffer utilization of different FQ schemes in a 10Gbps network. Proper buffer utilization is essential to maintaining low packet drop rates and ensuring the efficient use of network resources, so the evaluation allows us to assess how efficiently the available buffer space is used across different queueing mechanisms. We then adjust the load to evaluate the performance and investigate the relationship between buffer utilization and packet loss rates. By varying the load, we can better understand the trade-offs between maximizing buffer usage and minimizing packet

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

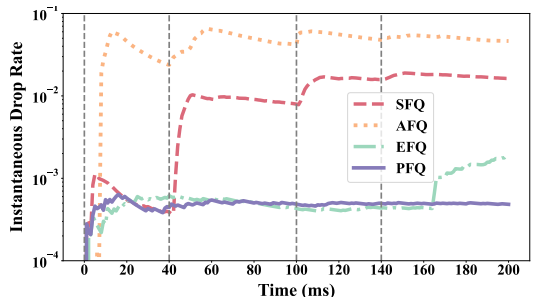


Fig. 10. The variation in packet drop rates under incast traffic for different FQ mechanisms. The gray dashed line indicates the time at which incast occurs.

loss, providing valuable insights into the performance of each scheme under different network conditions.

We monitor the instantaneous utilization of the buffer every 20 ms in Figure 8. The flow sizes follow a long-tailed distribution, dominated predominantly by mice flows. Other settings are kept unchanged. Results clearly illustrate that the EFQ consistently maintains a high utilization rate. Upon the arrival of large flows benefiting from the FBA mechanism, EFQ increases the allocation for the large flow, allowing subsequent packets to enter the buffer and queue. PFQ minimizes buffer waste allocated to mice flows through its recycling mechanism. The utilization rates of SFQ and AFQ are relatively low. In particular, AFQ applies excessively strict conditions for packet enqueuing, leading to the discard of large flow packets even when the buffer has significant available space.

Figure 9 shows that AFQ and SFQ consistently suffer from the high drop rate. The lines of EFQ and PFQ illustrate that the buffer capacity is underutilized in low-load conditions, leading to unnecessary packet loss. Subsequently, the packet loss rate decreases as buffer utilization increases. Nonetheless, once buffer utilization reaches a high level, it hinders the enqueuing of subsequent data packets, leading to a rise in packet loss rates. Therefore, to reduce the packet loss rate of FQ, it is necessary to maintain buffer utilization within an appropriate range (the gray shaded area).

E. Incast Performance

In this experiment, we investigate the performance of different FQs when confronted with incast traffic patterns. In Figure 10, the *y-axis* is on a logarithmic scale, representing the packet loss rate from the start of the simulation to the corresponding time. Incast occurs at 0ms, 40ms, 100ms, and 140ms. And we set the random packet loss to 0.05%.

Figure 10 shows that the packet loss rate of AFQ remains consistently high. As mentioned in Section II-B1, many flows are assigned to a small set of centralized physical queues, resulting in severe packet loss due to insufficient space in those queues. Similarly, SFQ exhibits a marked rise in packet loss rate when incast events occur. EFQ initially handles bursty traffic by utilizing the lenient buffer management. However, once the buffer reaches its capacity (around 160 ms), newly arrived packets are unable to be enqueued due to the lack of available buffer space. Notably, the packet loss rate in PFQ remains stable during the incast events. The underlying cause of this result lies in PFQ's proactive allocation of sufficient buffer space to accommodate incast traffic.

F. Convergence

To evaluate the convergence of traffic, we run five TCP flows in the dumbbell topology. The bandwidth is set to 10 Gbps. By analyzing how quickly and fairly the bandwidth is allocated to these flows, we aim to understand the efficiency and fairness of each scheme under dynamic and changing traffic conditions. Each flow starts in sequence at 10 ms intervals, and we monitor the instantaneous throughputs of five flows in Figure 11. Each line in the figure represents a flow.

Experiment results show that when five flows sequentially enter the traffic, PFQ shows good convergence speed in fair bandwidth allocation. The main reason lies in the design of PFQ, where the proactive allocated buffer space provides a fair and ample distribution to flows that arrive later. Although the five flows in the AFQ ultimately converge, the convergence process is slower, with noticeable fluctuations along the way. SFQ uses a hashing function to randomly map flows to queues, which can lead to uneven traffic distribution and queue imbalances, especially when elephant flows share a queue with smaller ones. This results in competition for limited buffer space, causing packet drops and throughput fluctuations. The round-robin scheduling of SFQ further exacerbates delays for smaller flows, making it less resilient to traffic bursts and variability. The flows in EFQ fail to achieve convergence, with the earlier-arriving flows occupying a larger share of the bandwidth. EFQ continuously increases the flow quota whenever buffer space is available. In contrast to other FQ mechanisms, elephant flow experiences much less restriction on the enqueuing of their subsequent packets.

G. Ablation experiment

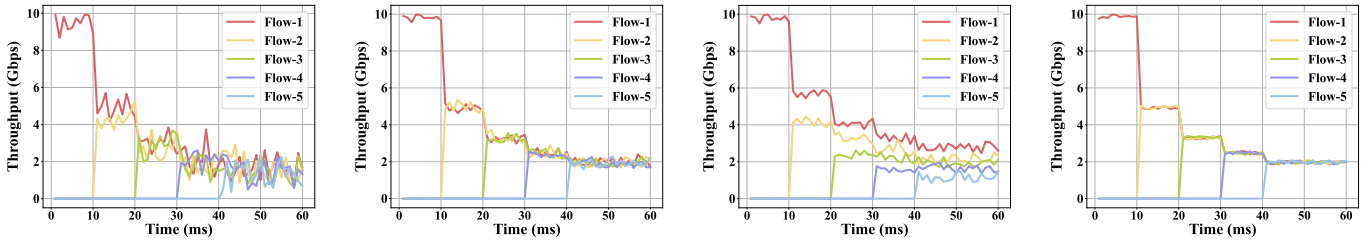
To evaluate the effectiveness of the recycling module and virtual queues, we conducted ablation experiments to provide comprehensive validation. We generated 2,000 flows under three different conditions: 10Gbps, 15Gbps, and 20Gbps.

Recycling Module. Figure 12(a) shows the *buffer utilization* and demonstrates that PFQ achieves improved buffer utilization compared to PFQ without recycling module across three scenarios, with improvements of 5.4%, 6.7%, and 10.8%, respectively. The recycling system effectively reclaims buffer space from small or inactive flows and reallocates it to large flows, minimizing the waste of allocated buffers. This improvement is directly reflected in throughput. Figure 12(b) illustrates the *normalized throughoput*, where PFQ achieves approximately a 4% increase in throughput.

Virtual Queues. If the PFQ assigns packets to physical queues based on the current length of each flow (*i.e.*, PFQ without virtual queues), it results in an uneven distribution of packets among the queues. As new packets arrive, queues with more packets are prone to encounter packet loss. Figure 12(c) presents the *drop rate* and the *y-axis* is on a logarithmic scale. It shows the implementation of virtual queues alleviates this issue, reducing the rate from 6.5% to 0.02% ($\sim$230$\times$) in

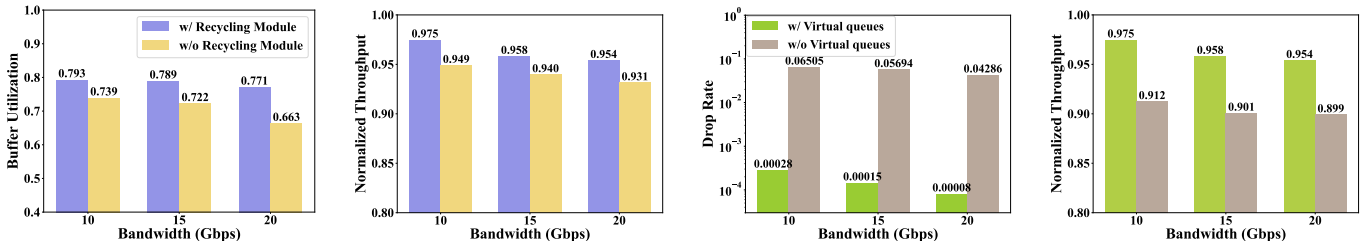
Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564



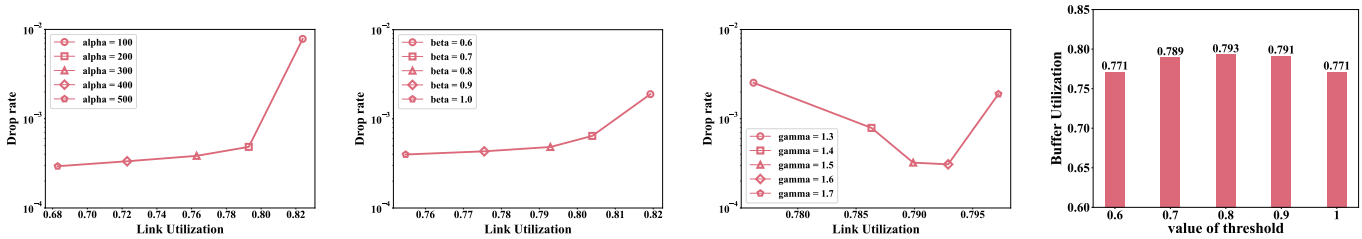
(a) SFQ (b) AFQ (c) EFQ (d) PFQ

Fig. 11. Variation of the throughput as five flows gradually join the network at an interval of 10ms.



(a) Buffer Utilization (b) Normalized Throughput (c) Drop Rate (d) Normalized Throughput

Fig. 12. Ablation experiment of recycling module and virtual queues.



(a) α (b) β (c) γ (d) *threshold*

Fig. 13. The PFQ's performance under different values of α, β, γ and *threshold*.

a 10Gbps environment. Moreover, Figure 12(d) demonstrates that a more balanced buffer allocation and lower packet loss contribute to the improvement of throughput performance.

H. Analysis of hyperparameters

In this section, we conduct the sensitivity analysis for the proactive buffer function factor. We conducted experiments in a 10Gbps environment. As shown in Figure 13, the relatively low α and β determine that the proactive buffer is small, allowing more buffer space to be allocated. As the values increase, buffer utilization decreases; however, a larger proactive buffer enhances the ability to manage incast scenarios. Thus, to obtain a good tradeoff between buffer efficiency and good reaction to incast traffic (the $y-axis$ is *drop rate under incast traffic*), we set α as 200 and β as 0.7.

We also analyze the recycling function factor. The experimental setup remains unchanged. As shown in Figure 13(c), PFQ recycles more buffers as γ increases, which improves buffer utilization. As for the drop rate, the buffers of inactive and small flows fail to effectively flow into large flows when γ is too low, leading to buffer shortage and packet loss in large flows. Also, when γ is too high, too many buffers are recycled, and subsequent packets of this flow will experience packet loss. Figure 13(d) presents that an excessively high or low value of *threshold* leads to misclassification of elephant and mice flows. To ensure the good performance of the recycling module, we set γ to 1.6 and *threshold* to 0.8.

TABLE III
IMPACT OF CYCLE LENGTH ON PERFORMANCE, FAIRNESS, AND OVERHEAD

T	Median FCT (μs)	Jain Fairness	Drop Rate	Overhead
0.1 ms	91.2	0.82	1.1%	38.6%
0.5 ms	89.8	0.83	0.9%	17.5%
1 ms	89.5	0.83	0.9%	11.2%
5 ms	89.7	0.83	0.9%	9.0%
10 ms	90.1	0.82	1.0%	7.8%
20 ms	**90.3**	**0.82**	**1.0%**	**6.7%**
30 ms	93.1	0.80	1.8%	6.3%
50 ms	98.6	0.76	3.4%	4.9%

As shown in Table III, we also analyze the impact of the hyperparameter T—the cycle interval for buffer reallocation and flow-to-queue remapping—on PFQ's performance. A natural concern is that the default value of $T = 20$ ms appears coarse relative to microsecond-scale RTTs in data centers. To address this, we evaluate PFQ under varying $T \in \{0.1, 0.5, 1, 5, 10, 20, 30, 50\}$ ms in the dumbbell topology. The results show that PFQ maintains consistently low

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

short-flow FCT (< 91 µs), high fairness (Jain index ≥ 0.82), and near-zero packet loss ($< 1.1\%$) for all $T \leq 20$ ms. This robustness stems from PFQ's design principle: it operates at the flow-aggregate level, not per-packet, and only needs to observe sufficient per-flow bytes over a period to distinguish active elephant flows from short or inactive ones. Performance begins to degrade at $T = 30$ ms (FCT $= 93.1$ µs, fairness $= 0.80$, drop rate $= 1.8\%$) and worsens significantly at $T = 50$ ms, as newly arriving flows suffer prolonged unfairness before the next remapping. Crucially, the CPU overhead at $T = 20$ ms is only 6.7% of a single core, making it highly deployable on programmable switches. Thus, $T = 20$ ms represents a conservative yet efficient trade-off: it ensures responsiveness to incast bursts while minimizing control-plane overhead, and remains well-aligned with the millisecond-scale dynamics of real-world flow arrivals in data centers.

V. Implementation Analysis

A. Additional cost

The Greedy algorithm for mapping flows to virtual queues operates on a small number of aggregate counters (equal to the number of virtual queues, typically < 8), and simply selects the queue with the minimum recent load. This decision is lightweight and well within the capabilities of existing programmable switch architectures. Furthermore, this mapping occurs once per flow per period, not on every packet, making the cost negligible in practice.

B. Feasibility of implementation in switch hardware

Here we leverage Tofino as an example of programmable switches.

1) **Alignment with Real-World Switch Architectures**: In current data center programmable switches, the total buffer capacity is typically around 480 Mbits, with SRAM distributed across each pipeline stage (10Mb per stage). TCAMs, used for flow matching and classification, provide around 6.25 MB per pipeline. These resources are shared across multiple queues, with each port supporting up to 8 priority queues. And a common setup uses approximately 22MB of SRAM for queuing, with each cell being 80 bytes. These queues are used to handle flow scheduling, where the PFQ approach would dynamically adjust buffer allocation based on flow characteristics. Flow state in PFQ is tracked efficiently, with each flow mapped to one of these queues. The state is compact and only stores necessary information (e.g., flow size, usage), reducing memory usage. The mapping mechanism (using flow IDs and periodic remapping) ensures low collision rates by limiting the number of entries that need to be checked, and the state is kept small to fit within switch memory limits.
2) **Resource Consumption Within Acceptable Limits**: Implementing PFQ requires maintaining state information for each flow, including metrics such as packet size and buffer utilization over a defined period. Since programmable switches like Tofino can use registers to store state, each flow's state could be mapped using a hash function to ensure efficient access. However, the register access limitations in programmable switches, specifically that a packet traversing a pipeline stage can only perform one type of access per cycle. This constraint could potentially limit certain use cases. To address this, our PFQ model is designed to store only the data from the previous cycle for each flow. This approach significantly reduces the need for multiple register accesses within a single cycle, aligning with the access constraints of programmable switches.
3) **Deployability of Control Logic on Programmable Switches**: The control logic integral to PFQ involves dynamic buffer allocation and determination of packet dequeue sequences based on real-time calculations, including addition, multiplication, and division operations. Programmable switches like Tofino are equipped with the necessary computational resources to handle such arithmetic operations efficiently within their data planes. This capability facilitates the real-time processing required by PFQ without introducing significant latency or computational overhead. Moreover, the P4 programming language, supported by Tofino, enables the expression of sophisticated packet-processing algorithms, including the arithmetic computations essential for PFQ's functionality.

VI. Related work

Fair queueing. Historically, FQ has attracted considerable attention and has been studied extensively. The original proposal Nagle [2] aims to achieve per-flow fair bandwidth allocation by using separate FIFO queues. And an effective bit-by-bit round robin (BR) algorithm [3] calculates a bid number to estimate each packet's departure time and prioritizes the transmission of packets with the lowest bid number using a priority queue. However, its implementation is highly challenging due to its complexity and considerable overhead.

Many algorithms have been proposed to reduce the complexity and cost of implementing fair queueing. Stochastic Fair Queueing (SFQ) [5] introduces the idea of hashing flows into individual physical queues. Based on SFQ, an efficient packet scheduling algorithm using DRR was proposed in [6]. However, significant performance degradation occurs when the number of flows far exceeds the number of queues. Several algorithms, such as FRED [7], RED-PD [8], and AFD [9], propose probabilistic packet dropping. They enforce fairness by dropping packets of flows that exceed the fair share rate.

Core-Stateless Fair Queueing (CSFQ) [15] achieves fair allocation by dividing the scheme into edge and core components. The complex rate estimation and packet labeling are managed at the edge network, allowing the core network to focus on simple packet forwarding based on these labels. Also, it drops packets with probability at the core based on the estimated fair rate. HCSFQ [16] generalizes CSFQ to a multi-level hierarchy, which is able to approximate hierarchical fair queueing accurately.

With the rapid advancement of programmable switches, some models have shown that FQ is now feasible to be approximated on programmable ASICs. AFQ [17] approximates BR

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

by using a few queues and the Rotating Strict Priority (RSP) Scheduler. It stores per-flow states in a count-min sketch [36]. EFQ [18], [19] equally assigns the amount of buffer space in each queue for each active flow. Moreover, EFQ introduces an FBA mechanism that adjusts buffer quotas in different queues according to the residual buffer space to improve buffer utilization. To consider the shared-memory architecture of commodity switches, A^2FQ [20] dynamically adjusts the number of effective queues according to traffic characteristics. In addition, it reveals the intrinsic trade-off between buffer utilization and fairness.

Programmable switches The programmable switch typically comprises three main components: packet processor, traffic manager (TM), and interfaces [37]. The TM is the most relevant to FQ. It is organized as a fixed number of FIFO physical queues per output port. The TM is responsible for two tasks: (*i*) *buffering*, which determines the appropriate output port and the FIFO queue for each packet, corresponding to the *Enqueue Module* in FQ, and (*ii*) *scheduling*, which selects a particular packet from the queue for transmission, corresponding to the *Dequeue Module* in FQ [1].

In recent years, many leading switching chip vendors have proposed programmable switching ASICs (*e.g.*, Intel Tofino 2 [10], Arista 7170 Series [11], Broadcom Trident 4 [12] and Jericho 2 [13], Cisco Silicon One [14]). These programmable switch ASICs can be programmed using high-level languages that define the specific packet processing required for each switch application. In this way, not only does it enhance the efficiency of the packet pipeline by restricting the protocols supported, but it also provides the flexibility to implement new or customized protocols as needed. In addition, programmable switch ASICs enable complete customized flexibility without significantly sacrificing performance, increasing power consumption, or adding costs [38].

Due to the rapid development of programmable switches, some programmable packet schedulers have been proposed in the past few years. Network operators could customize scheduling algorithms based on these schedulers to different requirements. PIFO [39] provides a hardware scheme to realize the abstraction of a push-in first-out (PIFO) queue. It relies on a tree of PIFO queues to maintain the scheduling order or time. Unlike PIFO, which only permits dequeueing from the head of the list, PIEO [40] allows dequeuing from arbitrary positions in the queue by enabling programmable predicate-based filtering. SP-PIFO [41] closely approximates the behavior of PIFO queues using strict-priority queues. Compared to PIFO, SP-PIFO can scale to large amounts of flows and quickly adapt to dynamic traffic. AIFO [21] utilizes only a single queue and achieves a bounded performance compared to PIFO. AIFO uses a sliding window to monitor the ranks of recent packets and calculates the relative rank of an incoming packet for admission control.

VII. Conclusion

In this paper, we present PFQ, a novel method aimed at achieving both fair bandwidth allocations and high network utilization in data center networks. Precisely, PFQ utilizes a buffer space reservation mechanism to manage incast traffic occurrences effectively. To improve link utilization while guaranteeing fairness, PFQ dynamically allocates and recycles buffers. Moreover, PFQ employs a greedy algorithm to periodically perturb the mapping of flow to virtual queues. Extension experiments on NS3 have shown that PFQ can achieve stable high fairness and network utilization simultaneously under different scenarios. We believe PFQ provides a new perspective on fair bandwidth allocation for data center networks.

Acknowledgments

This work is supported by the Project of the Department of Strategic and Advanced Interdisciplinary Research of Pengcheng Laboratory under grant No. 2025QYA001, the Major Key Project of PCL under grant NO. PCL2025A09, and the Shenzhen Key Lab of Software Defined Networking under grant No. ZDSYS20140509172959 98.

References

[1] N. K. Sharma, C. Zhao, M. Liu, P. G. Kannan, C. Kim, A. Krishnamurthy, and A. Sivaraman, "Programmable calendar queues for high-speed packet scheduling," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 685–699. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/sharma

[2] J. Nagle, "On packet switches with infinite storage," *IEEE Transactions on Communications*, vol. 35, no. 4, pp. 435–438, 1987.

[3] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," *SIGCOMM Comput. Commun. Rev.*, vol. 19, no. 4, p. 1–12, aug 1989. [Online]. Available: https://doi.org/10.1145/75247.75248

[4] S. Keshav, "On the efficient implementation of fair queueing," *Internetworking: Research and Experience*, vol. 2, no. 3, pp. 157–173, 1991.

[5] P. E. McKenney, "Stochastic fairness queueing," in *IEEE INFOCOM'90*. IEEE Computer Society, 1990, pp. 733–734.

[6] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round-robin," *IEEE/ACM Transactions on Networking*, vol. 4, no. 3, pp. 375–385, 1996.

[7] D. Lin and R. Morris, "Dynamics of random early detection," in *Proceedings of the ACM SIGCOMM '97 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM '97. New York, NY, USA: Association for Computing Machinery, 1997, p. 127–137. [Online]. Available: https://doi.org/10.1145/263105.263154

[8] "Controlling high-bandwidth flows at the congested router," in *Proceedings of the Ninth International Conference on Network Protocols*, ser. ICNP '01. USA: IEEE Computer Society, 2001, p. 192.

[9] R. Pan, L. Breslau, B. Prabhakar, and S. Shenker, "Approximate fairness through differential dropping," *SIGCOMM Comput. Commun. Rev.*, vol. 33, no. 2, p. 23–39, apr 2003. [Online]. Available: https://doi.org/10.1145/956981.956985

[10] Intel, "Intel Tofino 2," https://www.intel.com/content/www/us/en/products/details/network-io/intelligent-fabric-processors/tofino-2.html, 2024, [Online; accessed 02-August-2024].

[11] Arista, "Arista 7170 Series," https://www.arista.com/en/products/7170-series, 2024, [Online; accessed 02-August-2024].

[12] Broadcom, "Broadcom Trident 4," https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm56880-series, 2024, [Online; accessed 02-August-2024].

[13] ——, "Broadcom Jericho 2," https://www.broadcom.com/company/news/product-releases/52196, 2024, [Online; accessed 12-August-2024].

[14] Cisco, "Cisco Silicon One," https://www.cisco.com/site/us/en/products/networking/silicon-one/index.html, 2024, [Online; accessed 02-August-2024].

[15] I. Stoica, S. Shenker, and H. Zhang, "Core-stateless fair queueing: achieving approximately fair bandwidth allocations in high speed networks," *SIGCOMM Comput. Commun. Rev.*, vol. 28, no. 4, p. 118–130, oct 1998. [Online]. Available: https://doi.org/10.1145/285243.285273

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3661564

[16] Z. Yu, J. Wu, V. Braverman, I. Stoica, and X. Jin, "Twenty years after: Hierarchical {Core-Stateless} fair queueing," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 29–45.

[17] N. K. Sharma, M. Liu, K. Atreya, and A. Krishnamurthy, "Approximating fair queueing on reconfigurable switches," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. Renton, WA: USENIX Association, Apr. 2018, pp. 1–16. [Online]. Available: https://www.usenix.org/conference/nsdi18/presentation/sharma

[18] J. Liu, J. Huang, N. Jiang, W. Li, and J. Wang, "Achieving high utilization for approximate fair queueing in data center," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 2020, pp. 932–942.

[19] J. Liu, J. Huang, Z. Li, Y. Li, J. Wang, and T. He, "Achieving per-flow fairness and high utilization with limited priority queues in data center," *IEEE/ACM Trans. Netw.*, vol. 30, no. 5, p. 2374–2387, may 2022. [Online]. Available: https://doi.org/10.1109/TNET.2022.3172749

[20] D. Shan, G. Peng, S. Ren, J. Ma, S. Long, and Y. Tang, "Adaptive approximate fair queueing for shared-memory programmable switches," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 4, pp. 3563–3576, 2024.

[21] Z. Yu, C. Hu, J. Wu, X. Sun, V. Braverman, M. Chowdhury, Z. Liu, and X. Jin, "Programmable packet scheduling with a single queue," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, 2021, pp. 179–193.

[22] M. Alizadeh, S. Yang, M. Sharif, S. Katti, N. McKeown, B. Prabhakar, and S. Shenker, "pfabric: minimal near-optimal datacenter transport," in *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, ser. SIGCOMM '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 435–446. [Online]. Available: https://doi.org/10.1145/2486001.2486031

[23] W. Bai, L. Chen, K. Chen, D. Han, C. Tian, and H. Wang, "Information-agnostic flow scheduling for commodity data centers," in *Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'15. USA: USENIX Association, 2015, p. 455–468.

[24] L. Chen, K. Chen, W. Bai, and M. Alizadeh, "Scheduling mix-flows in commodity datacenters with karuna," in *Proceedings of the 2016 ACM SIGCOMM Conference*, ser. SIGCOMM '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 174–187. [Online]. Available: https://doi.org/10.1145/2934872.2934888

[25] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*, 2010, pp. 267–280.

[26] D. Shan, F. Ren, P. Cheng, R. Shu, and C. Guo, "Micro-burst in data centers: Observations, analysis, and mitigations," in *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, 2018, pp. 88–98.

[27] J. Jaffe, "Bottleneck flow control," *IEEE Transactions on Communications*, vol. 29, no. 7, pp. 954–962, 1981.

[28] L. Brown, Y. Kothari, A. Narayan, A. Krishnamurthy, A. Panda, J. Sherry, and S. Shenker, "How i learned to stop worrying about cca contention," in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, ser. HotNets '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 229–237. [Online]. Available: https://doi.org/10.1145/3626111.3628204

[29] G. Zeng, W. Bai, G. Chen, K. Chen, D. Han, Y. Zhu, and L. Cui, "Congestion control for cross-datacenter networks," in *2019 IEEE 27th International Conference on Network Protocols (ICNP)*, 2019, pp. 1–12.

[30] G. Zeng, J. Qiu, Y. Yuan, H. Liu, and K. Chen, "Flashpass: Proactive congestion control for shallow-buffered wan," in *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, 2021, pp. 1–12.

[31] Z. Guan, W. Li, X. He, S. Zhang, and K. Li, "dbfc: Destination-based backpressure flow control for incast," in *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 1853–1860.

[32] K. B. Nougnanke, Y. Labit, M. Bruyere, S. Ferlin, and U. Aïvodji, "Learning-based incast performance inference in software-defined data centers," in *2021 24th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*, 2021, pp. 118–125.

[33] NS-3, "ns-3.28," https://www.nsnam.org/releases/ns-3-28/, 2024, [Online; accessed 27-December-2024].

[34] H. Zhang, J. Zhang, W. Bai, K. Chen, and M. Chowdhury, "Resilient datacenter load balancing in the wild," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 253–266. [Online]. Available: https://doi.org/10.1145/3098822.3098841

[35] R. K. Jain, D.-M. W. Chiu, W. R. Hawe *et al.*, "A quantitative measure of fairness and discrimination," *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA*, vol. 21, p. 1, 1984.

[36] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[37] Broadcom, "Silicon innovations in programmable switch hardware," https://www.broadcom.com/blog/silicon-innovations-in-programmable-switch-hardware, 2024, [Online; accessed 01-August-2024].

[38] D. Scholz, H. Stubbe, S. Gallenmüller, and G. Carle, "Key properties of programmable data plane targets," in *2020 32nd International Teletraffic Congress (ITC 32)*. IEEE, 2020, pp. 114–122.

[39] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proceedings of the 2016 ACM SIGCOMM Conference*, ser. SIGCOMM '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 44–57. [Online]. Available: https://doi.org/10.1145/2934872.2934899

[40] V. Shrivastav, "Fast, scalable, and programmable packet scheduler in hardware," in *Proceedings of the ACM special interest group on data communication*, 2019, pp. 367–379.

[41] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out behaviors using Strict-Priority queues," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 59–76. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/alcoz

Yike Wang. Yike Wang received his B.S. degree in the Department of Computer Science and Technology from Northeastern University in 2023. Now he is a graduate student in Tsinghua Shenzhen International Graduate School, Tsinghua University. His research interests include traffic scheduling and congestion control.

Qing Li. Qing Li received the B.S. degree (2008) from Dalian University of Technology, Dalian, China, the Ph.D. degree (2013) from Tsinghua University, Beijing, China; both in computer science and technology. He is currently a full professor at Peng Cheng Laboratory, Shenzhen, China. His research interests include reliable and scalable routing of the Internet, software defined networking, network function virtualization, in-network caching/computing, edge computing, traffic scheduling, transmission control, video delivery, etc.

Feixue Han. Feixue Han received her B.S. degree from Beijing University in 2018. She is currently working toward her Ph.D. degree in the Department of Computer Science and Technology at Tsinghua University. Her current research interests include congestion control and traffic scheduling.

Changlin Jiang. Changlin Jiang received the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, China, in 2015. He is currently a Senior Engineer with Pengcheng Laboratory, Shenzhen, China. His main research interests include in-network computing, networks for AI, etc.

Yuan Yang. Yuan Yang received the B.Sc., M.Sc., and Ph.D. degrees from Tsinghua University, where he is currently an Associate Researcher with the Department of Computer Science and Technology. He was a Visiting Ph.D. Student with The Hong Kong Polytechnic University. His major research interests include computer network architecture and routing protocols.

Rui Huang. Rui Huang received his B.S. degree in Software Engineering from Beihang University in 2024. He is currently pursuing an M.S. degree in Computer Science and Technology at Tsinghua University. His research interests include data center networks and high-performance networking.

Yong Jiang. Yong Jiang received the B.S. degree and the Ph.D. degree from Tsinghua University, Beijing, China, both in computer science and technology. He is currently a full professor at the Tsinghua Shenzhen International Graduate School. His research interests include the future network architecture, the Internet QoS, network function virtualization, etc.

Mingwei Xu. Mingwei Xu received the B.S. and Ph.D. degrees from Tsinghua University, Beijing, China, where he is currently a Full Professor with the Department of Computer Science. He is the Winner of National Science Foundation for Distinguished Young Scholars of China. He is an Executive Director of China Institute of Communications. His research interests include Internet architecture, Internet routing, and Cybersecurity.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on March 01,2026 at 12:45:33 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.